UNIT KOMPETENSI 06 • TI.ST.06.2026

MENGIMPLEMENTASIKAN PEMROGRAMAN TERSTRUKTUR

Implementasi Riil pada Proyek: Portal Karir, Profil Industri & Modul Automation-GPT (Fullstack Web)

Presenter / Mahasiswa	Aditya Ristanto Mukti (Fullstack Web Developer Intern)	**Repository**	`github.com/AdityaRistantoMukti/Website_Intersis`
Institusi / Mitra	PT Global Philsonic Technologies (Intersis) Batam	**Tech Stack Utama**	React 19, Vite, Tailwind CSS v4, Node.js Express 5, MySQL2
Cakupan Outline Sub-Title	Penguasaan tipe data • Percabangan • Perulangan • Prosedur/Fungsi • Array • File I/O		

I. PETA OUTLINE & STRUKTUR PEMBAHASAN

Unit kompetensi **Mengimplementasikan Pemrograman Terstruktur** ini disusun secara berkesinambungan mencakup seluruh sub-judul yang didefinisikan pada diagram alur kompetensi, diaplikasikan langsung pada arsitektur website yang telah dibangun:

- **Penguasaan tipe data**: Mengelola tipe data primitif dan referensi (String, Number, Boolean, Array, Object) serta validasi skema payload.
- **Percabangan**: Menerapkan logika kondisional if-else, guard clauses, switch-case evaluasi role, dan ternary expressions.
- **Perulangan**: Mengoperasikan loop deklaratif dan imperatif (map, filter, reduce, forEach) untuk manipulasi koleksi data.
- **Prosedur/Fungsi**: Menyusun pure functions, higher-order functions, fungsi asinkron (async/await), dan pipeline middleware.
- **Array**: Melakukan transformasi data tak bermutasi (immutable), destructuring array, dan operasi spreading.
- **File I/O**: Mengeksekusi pembacaan, penulisan, dan pembersihan berkas media asinkron melalui modul Node.js 'fs'.

II. NASKAH WICARA LENGKAP PRESENTER (VERBATIM SPEECH SCRIPT)

🎤 PANDUAN NARASI WICARA PRESENTER (SPEAKING SCRIPT)

[Pembukaan & Sikap] "Selamat pagi/siang Dewan Penguji yang terhormat. Fondasi dari setiap aplikasi perangkat lunak yang handal bertumpu pada algoritma terstruktur yang solid. Pada unit **Mengimplementasikan Pemrograman Terstruktur**, saya akan memaparkan penerapan 6 pilar pemrograman terstruktur pada proyek Website Intersis."

[Menjelaskan Sub 1: Penguasaan Tipe Data] "Pertama, *Penguasaan Tipe Data*. Kami membedakan secara ketat antara tipe data primitif (String, Number, Boolean) dan referensi (Array, Object). Di backend, kami menggunakan pustaka `express-validator` untuk memastikan tipe data yang masuk dari request body benar-benar valid, seperti sanitasi string email, konversi integer untuk limit pagination, dan validasi boolean untuk status aktif."

[Menjelaskan Sub 2: Percabangan] "Kedua, *Percabangan (Conditionals)*. Untuk menghindari 'callback hell' dan nesting kondisi yang berlebihan (arrow anti-pattern), kami menerapkan **Guard Clauses** (early return pattern). Contohnya pada autentikasi: jika token tidak ada, fungsi langsung mengembalikan status 401 dan berhenti mengeksekusi baris berikutnya. Kami juga menggunakan `switch-case` untuk pemetaan izin multi-role (administrator, HRD, staff) dan operator ternary pada rendering komponen JSX."

[Menjelaskan Sub 3, 4, & 5: Perulangan, Prosedur/Fungsi, dan Array] "Ketiga, keempat, dan kelima adalah *Perulangan, Fungsi, dan Array*. Kami konsisten menggunakan paradigma fungsional modern. Kami memanfaatkan `Array.prototype.map()` untuk merender daftar kartu lowongan secara deklaratif di React, `.filter()` untuk menyaring data berdasarkan kategori yang dipilih, dan `async/await` pada setiap prosedur I/O untuk menjaga eksekusi asynchronous non-blocking."

[Menjelaskan Sub 6: File I/O] "Keenam, *File I/O (Input/Output)*. Saat admin memperbarui foto profil korporat atau menghapus item dari galeri, sistem kami tidak hanya menghapus referensi di database, tetapi juga mengeksekusi operasi file I/O menggunakan `fs.unlinkSync` untuk menghapus file fisik di disk server. Dengan demikian, kapasitas penyimpanan server tetap bersih dari sampah file yatim (orphan files)."

III. PEMBAHASAN TEKNIS RINCI & BEDAH KODE NYATA PROYEK

Berikut adalah pembuktian implementasi teknis dari masing-masing sub-judul berdasarkan baris kode dan berkas nyata di dalam repository proyek:

6.1 & 6.2. Validasi Tipe Data & Pola Percabangan Guard Clauses

Kaitkan ke Codebase: **`backend/src/middlewares/authMiddleware.js`**

Pola Guard Clause diterapkan untuk menyederhanakan alur percabangan dan menjamin keamanan rute sebelum data diproses lebih lanjut:

backend/src/middlewares/authMiddleware.js

```
// Penerapan Guard Clauses: Mencegah Nested If Berlebihan
const verifyToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];

  // Guard 1: Periksa keberadaan header
  if (!authHeader) {
    return res.status(401).json({ success: false, message: 'Akses ditolak: Token tidak ditemukan.' });
  }

  // Guard 2: Periksa format Bearer
  const token = authHeader.split(' ')[1];
  if (!token) {
    return res.status(401).json({ success: false, message: 'Format token tidak valid.' });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next(); // Lolos validasi, lanjutkan ke controller
  } catch (err) {
    return res.status(403).json({ success: false, message: 'Token kadaluarsa atau tidak sah.' });
  }
};
```

6.3, 6.4 & 6.5. Manipulasi Array Fungsional & Rendering Deklaratif React

Kaitkan ke Codebase: **`frontend/src/pages/public/CareerPage.jsx`**

Operasi array immutable (map dan filter) digunakan secara terpadu untuk menyaring lowongan kerja berdasarkan pencarian teks dan kategori tanpa memutasi data asli:

frontend/src/pages/public/CareerPage.jsx

```
// Filtering Multi-Parameter & Transformasi Array Deklaratif
const filteredJobs = useMemo(() => {
  return jobs.filter((job) => {
    const matchesSearch = job.title.toLowerCase().includes(searchTerm.toLowerCase()) ||
                          job.description.toLowerCase().includes(searchTerm.toLowerCase());
    const matchesCategory = selectedCategory === 'all' || job.category_id === Number(selectedCategory);
    return matchesSearch && matchesCategory;
  });
}, [jobs, searchTerm, selectedCategory]);

// Render Array ke Elemen JSX
return (
  <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
    {filteredJobs.map((job) => (
      <JobCard key={job.id} jobData={job} onSelect={() => setSelectedJob(job)} />
    ))}
  </div>
);
```

6.6. Operasi File I/O & Pembersihan Berkas Fisik di Server

Kaitkan ke Codebase: **backend/src/controllers/admin/adminContentControllers.js**

Sistem mengelola berkas media fisik di storage lokal backend menggunakan modul inti `fs` dan `path`:

backend/src/controllers/admin/adminContentControllers.js

```
const fs = require('fs');
const path = require('path');

// File I/O: Menghapus File Gambar Fisik dari Disk Saat Record Dihapus
exports.deleteGalleryItem = async (req, res, next) => {
  try {
    const { id } = req.params;
    const [rows] = await db.query('SELECT image FROM gallery WHERE id = ?', [id]);

    if (rows.length > 0 && rows[0].image) {
      const filePath = path.join(__dirname, '../../../uploads', rows[0].image);
      // Validasi keberadaan file sebelum penghapusan
      if (fs.existsSync(filePath)) {
        fs.unlinkSync(filePath); // Eksekusi penghapusan file dari filesystem
      }
    }

    await db.query('DELETE FROM gallery WHERE id = ?', [id]);
    res.json({ success: true, message: 'Item galeri dan berkas fisik berhasil dihapus.' });
  } catch (err) {
    next(err);
  }
};
```

IV. POIN KUNCI PRESENTASI (SLIDE TALKING POINTS)

POINTER RINGKAS UNTUK TAMPILAN SLIDE / LAYAR

- Tipe Data & Skema: Validasi ketat tipe primitif dan referensi dengan express-validator.
- Guard Clauses Pattern: Mengurangi nesting if-else untuk meningkatkan keterbacaan kode.
- Operasi Array Deklaratif: Pemanfaatan .filter() dan .map() immutable untuk reaktifitas UI.
- Fungsi Asinkron: Penggunaan async/await untuk penanganan I/O non-blocking.
- Manajemen File I/O: Otomasi penghapusan file fisik via fs.unlinkSync untuk mencegah storage leak.

V. MATRIKS TANYA JAWAB & PANDUAN PEMBELAAN SIDANG (Q&A DEFENSE)

Prediksi pertanyaan evaluasi dari Penguji / Asesor / Pembimbing Industri beserta jawaban teknis yang siap dipertahankan:

Q1: Apa perbedaan manipulasi array imperatif (for loop biasa) dengan fungsional (.map/.filter) dalam konteks React?

Jawaban Pembelaan Presenter: Metode fungsional seperti .map() dan .filter() bersifat declarative dan immutable (menghasilkan array baru tanpa mengubah data asal), yang sangat krusial dalam siklus hidup React untuk memicu re-render secara akurat berdasarkan perbandingan referensi (shallow comparison).

Q2: Mengapa Anda memeriksa fs.existsSync() sebelum menjalankan fs.unlinkSync()?

Jawaban Pembelaan Presenter: Jika kita memanggil fs.unlinkSync pada path file yang sudah tidak ada di disk (misalnya file terhapus manual oleh admin server), Node.js akan melempar ENOENT exception yang dapat menyebabkan kegagalan request. Validasi keberadaan file menjamin operasi penghapusan record database tetap sukses.